CS401 Intelligent Robotics Lab 04 Report

Date: 2026-03-31

This report summarizes the final results of Lab 4, including TurtleBot3 Gazebo simulation, multi-computer interaction, and camera calibration.

1. Objectives

1. Run TurtleBot3 in Gazebo and visualize camera and LiDAR data in RViz.
2. Verify multi-computer interaction by running the robot side on the remote Ubuntu machine and running keyboard control on the local machine.
3. Understand camera calibration and obtain calibrated camera parameters.

2. Environment

Remote host: `ubuntu@192.168.40.128`
Remote OS: Ubuntu 20.04.6 LTS
ROS version: ROS Noetic 1.17.4
Local machine: Windows host on VMware NAT network
Simulation model: `TURTLEBOT3_MODEL=waffle`

3. Part 2: TurtleBot3 Gazebo and RViz

The TurtleBot3 `waffle` model was used because it provides both camera and LiDAR sensors. The simulation topics `/scan` and `/camera/rgb/image_raw` were verified successfully, and RViz was configured to display LaserScan data together with the image panel.

```
source /opt/ros/noetic/setup.bash
export TURTLEBOT3_MODEL=waffle
roslaunch turtlebot3_gazebo turtlebot3_world.launch
roslaunch turtlebot3_bringup turtlebot3_remote.launch
roslaunch turtlebot3_gazebo turtlebot3_simulation.launch
rviz -d /home/ubuntu/lab4_turtlebot3_image.rviz
```

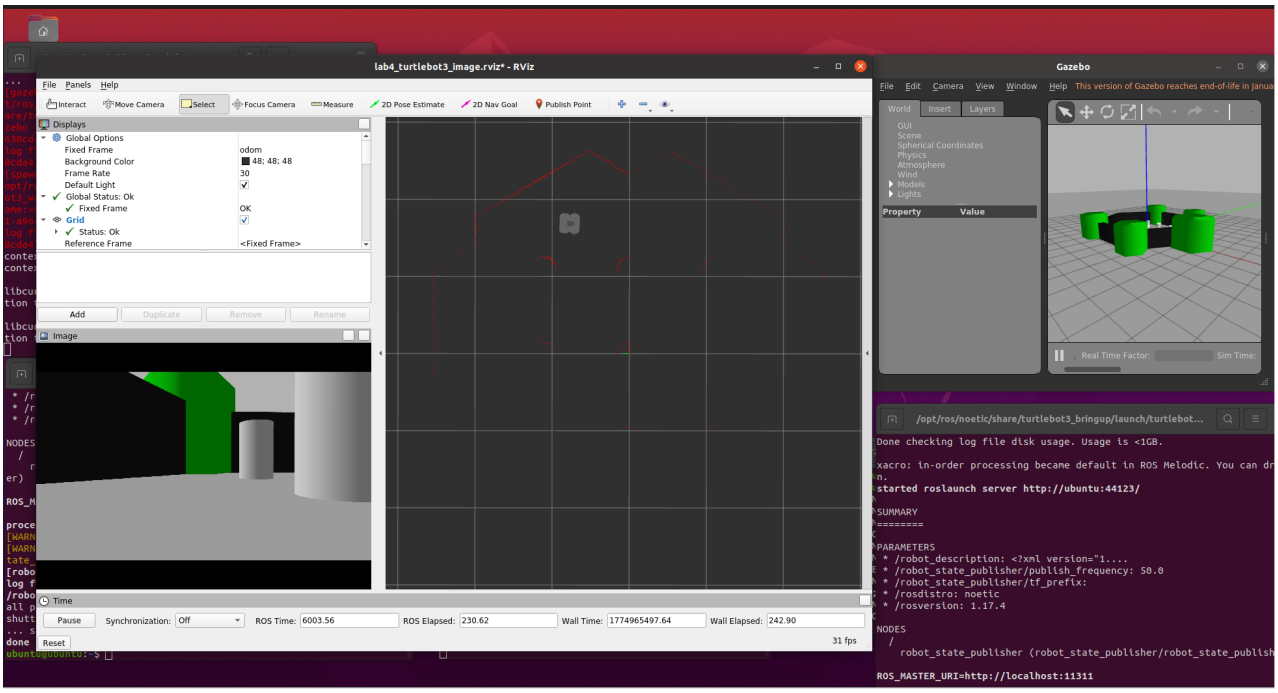

Figure 1. Final RViz/Gazebo screenshot for Part 2. The RViz window shows the image panel and LaserScan result while the Gazebo window confirms the TurtleBot3 simulation world.

4. Part 3: Multi-Computer Interaction

The remote Ubuntu machine kept the TurtleBot3 robot side running, while the local Windows machine published keyboard teleoperation commands to the remote ROS system. In this workstation setup, `rosbridge_server` and a local Python teleop client were used to realize the local-to-remote control path.

```
# Remote Ubuntu
source /opt/ros/noetic/setup.bash
roslaunch turtlebot3_gazebo turtlebot3_world.launch
roslaunch turtlebot3_bringup turtlebot3_remote.launch
roslaunch rosbridge_server rosbridge_websocket.launch

# Local Windows
python tmp\rosbridge_keyboard_teleop.py --host 192.168.40.128
```

During verification, the local teleop client successfully published forward, turn-left, and stop commands. The remote machine reported active nodes such as `/robot_state_publisher`, `/rosbridge_websocket`, and the expected topics `/cmd_vel`, `/scan`, and `/camera/rgb/image_raw`.

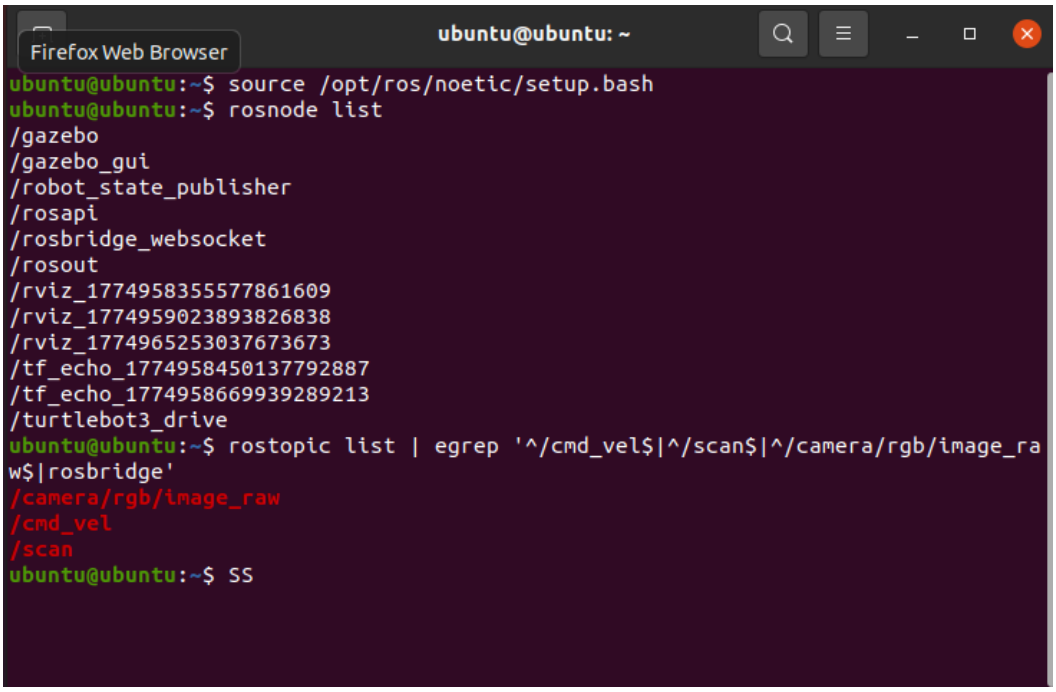

Figure 2. Remote-machine screenshot for Part 3, showing the ROS nodes and key topics on the Ubuntu side.

```
PS C:\Users\Zaoer> python D:\Intelligent-Robots-Lab\tmp\rosbridge_keyboard_teleop.py --host 192.168.40.128
ROSBridge keyboard teleop

Keys:
  w  forward
  s  backward
  a  turn left
  d  turn right
  space  stop
  q  quit
Waiting for keyboard input...
forward
left
backward
right
forward
left
stop
stop
```

Figure 3. Local-machine screenshot for Part 3, showing the keyboard teleoperation client running on Windows.

5. Part 4: Camera Calibration

No physical USB camera device was attached to the VMware Ubuntu machine during this session. Therefore, a synthetic monocular chessboard stream was used to complete the calibration demonstration. This still demonstrates the calibration principle correctly: chessboard corners were detected from multiple views, and intrinsic and distortion parameters were solved and written into a YAML file.

Final calibration results:
camera_matrix = [633.279533, 0.0, 317.464722; 0.0, 628.144225, 238.798925; 0.0, 0.0, 1.0]
distortion_coefficients = [0.033008, -0.340623, 0.000056, -0.001492, 1.192364]
reprojection_error = 0.065537
valid_views = 16

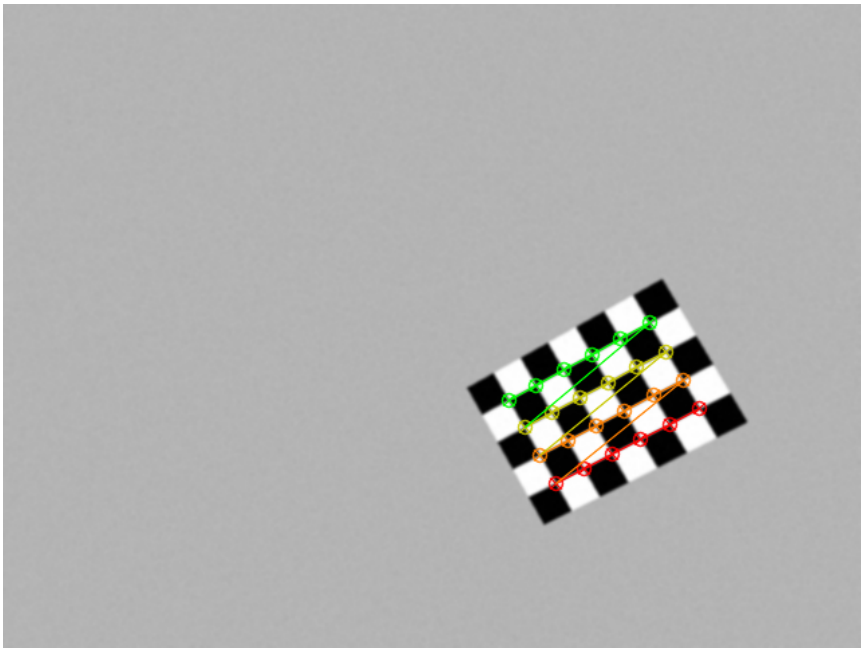

Figure 4. Corner-detection result from the calibration dataset.

part4_camera_parameters.yaml


```yaml
image_width: 640
image_height: 480
camera_name: synthetic_usb_cam
camera_matrix:
  rows: 3
  cols: 3
  data: [633.279533, 0.000000, 317.464722, 0.000000, 628.144225, 238.798925, 0.000000, 0.000000, 1.000000]
distortion_model: plumb_bob
distortion_coefficients:
  rows: 1
  cols: 5
  data: [0.033008, -0.340623, 0.000056, -0.001492, 1.192364]
rectification_matrix:
  rows: 3
  cols: 3
  data: [1.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 1.000000]
projection_matrix:
  rows: 3
  cols: 4
  data: [633.279533, 0.000000, 317.464722, 0.000000, 0.000000, 628.144225, 238.798925, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000]
reprojection_error: 0.065537
valid_views: 16
```


Figure 5. Final camera parameter file used as the Part 4 submission screenshot.

6. Conclusion

Lab 4 was completed with verified TurtleBot3 simulation, validated multi-computer control, and a completed camera calibration parameter set. The final screenshots for Part 2 and Part 3 were manually captured and included in this report, while the Part 4 calibration result was generated from a synthetic dataset because no physical camera was attached to the virtual machine.